

F215 Digital Design — Tutorial 5

Decoders, multiplexers, and their applications

Question 1 — Decoders as Minterm and Maxterm Generators

(a) Implement the following with an active-high decoder:

1. Design a 1-bit comparator: inputs X, Y; outputs $X > Y$ and $X = Y$. Use a single 2-to-4 decoder and one single OR gate.
2. $H(A,B,C) = A'C + AB'C' + ABC$, using a 3-to-8 decoder.
3. $F(X,Y) = \Sigma m(0,2,3)$, using a 2-to-4 decoder.
4. $G(A,B,C) = \Sigma m(0,1,3,6)$, using a 3-to-8 decoder.

(b) Implement the following with an active-low decoder:

1. $H(A,B,C) = B \cdot (A+B+C') \cdot (A'+B'+C)$, using a 3-to-8 decoder.
2. $F(X,Y) = \Pi M(1)$, using a 2-to-4 decoder.
3. $G(A,B,C) = \Pi M(2,4,5,7)$, using a 3-to-8 decoder.

(c) Building larger SOPs from smaller decoders (hierarchical):

1. Implement $F(W,X,Y,Z) = \Sigma m(1,4,7,9,10)$ using only 2-to-4 decoders and one OR gate.
-

Question 2 — Implementing Expressions with Multiplexers

(a) 3-variable functions with 8-to-1 mux

1. $H(A,B,C) = (A+B+C)(A+B'+C')(A'+B+C')$.
2. $F(A,B,C) = \Sigma m(0,2,5,7)$
3. $G(X,Y,Z) = \Sigma m(1,3,4,5,6)$

(b) 4-variable functions with 8-to-1 mux

1. $G(A,B,C,D) = \Sigma m(2,4,6,12,13,15)$ — select lines B, C, D (in that order), with A on the data lines.
2. $H(A,B,C,D) = \Sigma m(1,4,11,14)$, don't cares at 2,7 — select on A,B,C; D on the data lines.
3. $F(A,B,C,D) = \Sigma m(0,5,7,8,10,11,12,14,15)$ — select on A,B,C; D on the data lines.

(c) 4-variable function with 4-to-1 mux

1. $H(A,B,C,D) = \Sigma m(0,2,5,6,8,9,11,15)$ — select lines A, B.

(d) Building larger functions by cascading multiplexers:

1. Design the Sum output of a Full Adder (inputs X, Y, Cin) using only two 4-to-1 multiplexers.
-

Question 3 — Application Questions

1. Binary-to-Gray code conversion

Design a 3-bit Binary-to-Gray converter using only one 3-to-8 decoder and one 8-to-3 encoder.

2. Sharing one circuit across multiple functions

Three functions of the same 3 variables are needed simultaneously:

$$F1(A,B,C) = \Sigma m(1,3,5) \quad F2(A,B,C) = \Sigma m(3,5,6) \quad F3(A,B,C) = \Sigma m(1,6)$$

- (a) Implement all three using one shared 3-to-8 decoder plus one OR gate per function (as in the lecture's carry/sum example).
- (b) Of the 8 AND gates inside the decoder, how many correspond to minterms actually used by at least one of F1, F2, F3? How many are wasted?
- (c) Imagine an alternative circuit that replaces the decoder with a smaller AND-gate array — one AND gate for each distinct product term actually needed (not all 8 possible minterms) — where each AND gate's output can be connected to any subset of the OR gates that need it. Using your answer to (b), how many AND gates would this alternative circuit need for the same problem? Which AND gate(s) would need to feed more than one OR gate?

3. Driving a multi-digit display

A 4-digit display needs to show 4 independent BCD digits, but you only want to wire up one shared set of 7 segment-driver lines plus a shared BCD-to-7-segment decoder, rather than giving every digit its own.

- (a) If each digit had its own dedicated 7-segment decoder and drivers, how many segment-driver wires would you need in total for 4 digits? What's the minimum number of wires if all 4 digits share one set of 7 segment lines (plus some number of digit-enable lines)?
- (b) Suppose you have a small circuit that repeatedly cycles through the 2-bit values 00, 01, 10, 11 in sequence, moving to the next value at a steady fast rate — fast enough that the human eye can't perceive the switching. Design a demux (or decoder) driven by this repeating 2-bit sequence that enables exactly one of the 4 digits' common line at a time.
- (c) At every instant, a 4-to-1 mux — driven by the same repeating 2-bit sequence — selects which digit's 4-bit BCD value gets sent to the shared BCD-to-7-segment decoder. Sketch the complete system: the 2-bit sequence feeding both the mux (selecting a BCD value) and the demux/decoder (driving the digit-enable lines), with the mux's output feeding the shared 7-segment decoder and shared segment lines.
- (d) Why is it essential that the mux and the demux/decoder are driven by the exact same 2-bit value at every instant? What would go wrong if they drifted out of sync even slightly?